DEPLOYMENT GUIDE

TravelPort

Docker · GitHub Actions CI/CD · Server Provisioning

VERSION	PLATFORM	CI/CD	REGISTRY
v1.0.0	Docker + Linux VPS	GitHub Actions	Docker Hub

Table of Contents

Deployment Guide — TravelPort v1.0.0

1 Prerequisites

Required tools for local development and production

Local Development

- **Node.js 20+** — Frontend build
- **.NET 8 SDK** — Backend build & EF tools
- **SQL Server Express** — `localhost\SQLEXPRESS`
- **Docker Desktop** (optional) — for container testing
- **Git** — Source control

Production Server

- **Ubuntu 22.04+** (or any Linux)
- **Docker Engine 24+**
- **Docker Compose v2**
- **SSH access** — For GHA deployment
- **Ports open:** 80 (HTTP), 443 (HTTPS)

2 Docker Setup

Three-container stack: SQL Server + .NET API + Nginx/React

2.1 docker-compose Services

SERVICE	IMAGE	PORT	HEALTH CHECK
`sqlserver`	mcr.microsoft.com/mssql/server:2022-latest	1433 (internal)	sqlcmd SELECT 1
`api`	docker.io/{owner}/travelport-api	8080 (internal)	GET /health → 200
`web`	docker.io/{owner}/travelport-web	80 → host	HTTP 200 on /

The `web` (Nginx) container serves the React SPA and reverse-proxies `/api/*` requests to the `api` container on port 8080.

2.2 Environment Variables

Copy `.env.example` to `.env` and fill in the required values:

```
# Required
DB_SA_PASSWORD=YourStr0ngP@ssword!
JWT_SECRET=minimum-32-character-random-secret-key
JWT_ISSUER=TravelPort
JWT_AUDIENCE=TravelPortApp
ASPNETCORE_ENVIRONMENT=Production

# Email (Office365)
EMAIL_ENABLED=true
EMAIL_HOST=smtp.office365.com
EMAIL_PORT=587
EMAIL_USERNAME=noreply@yourdomain.com
EMAIL_PASSWORD=your-smtp-password

# Payment (Razorpay — optional)
RAZORPAY_ENABLED=false
RAZORPAY_KEY_ID=rzp_test_xxx
RAZORPAY_KEY_SECRET=your-secret
```

2.3 Build & Run

```
# First run — build images and start all services
cp .env.example .env
# Edit .env with your values
docker compose build
docker compose up -d

# Later runs — api/web use pull_policy: always, so docker compose up -d
# fetches the newest :latest images from Docker Hub before restart
docker compose up -d

# Verify services
docker compose ps
docker compose logs api --tail=50

# App URLs
# Frontend:  http://localhost
# API Swagger: http://localhost/api/swagger
```

First Start Timing

SQL Server takes ~45 seconds to initialize. The API container waits via a healthcheck dependency. EF Core migrations and data seeding run automatically on API startup — no manual steps required.

3 CI/CD Pipeline

GitHub Actions 3-job workflow

3.1 Workflow Jobs

1

build-and-verify

Triggered on push to Development or PR. Runs dotnet test + dotnet build for the backend and vitest + npm run build for the frontend. All checks must pass before proceeding.

2

docker-build-push

Runs only on push to Development (not on PRs). Requires manual approval, then builds multi-stage Docker images for API and Web and pushes tagged images to Docker Hub.

3

verify-images

Pulls the newly published API and Web images from Docker Hub to confirm they are available for downstream docker compose deployments.

â„¹ **Local Commit Rule**

Developers enable `git config core.hooksPath .githooks` once per clone so the shared repository test gate runs before every commit.

3.2 Required GitHub Secrets

SECRET NAME	VALUE
`DOCKERHUB_USERNAME`	Docker Hub namespace used for API and Web images
`DOCKERHUB_TOKEN`	Docker Hub access token with push permission
`DB_SA_PASSWORD`	SQL Server SA password (strong, 16+ chars)
`JWT_SECRET`	Minimum 32-character random string
`EMAIL_USERNAME`	SMTP sender email address
`EMAIL_PASSWORD`	SMTP password or app password

SECRET NAME	VALUE
`RAZORPAY_KEY_SECRET`	Razorpay Key Secret (optional)

> ⚠ **Security Warning**
>
> Never commit secrets to appsettings.json or any tracked file. Secrets flow via GitHub Secrets → server .env file at deploy time. The .env file is in .gitignore.

3.3 Rollback

```
# SSH into server and roll back to previous image tag
docker compose pull travelport-api:previous-tag
docker compose up -d

# Or use git tag to redeploy a previous release
git checkout v1.0.0
git push origin Development  # triggers pipeline
```

4 Manual Deployment

First-time server bootstrap

4.1 Server Bootstrap

```
# 1. Install Docker on Ubuntu
curl -fsSL https://get.docker.com | sh
sudo usermod -aG docker $USER

# 2. Clone repository
git clone https://github.com/YourOrg/Goibibo-AI-Assignment.git
cd Goibibo-AI-Assignment

# 3. Create .env from secrets
cat > .env <# ... all other vars
EOF

# 4. Start or restart pre-built images
# docker-compose.yml uses pull_policy: always for api/web
docker compose up -d
```

5 Database Management

5.1 Reset Database (Dev Only)

```
# Drop + recreate (when DataSeeder changes)
cd backend
dotnet ef database drop --project src/Persistence --startup-project src/API --force
dotnet ef database update --project src/Persistence --startup-project src/API
```

6 Troubleshooting

SYMPTOM	LIKELY CAUSE	FIX
API returns 500 on start	DB not ready yet	Wait 45–60 s for SQL Server healthcheck to pass
Login returns 401 immediately	JWT_SECRET mismatch between API instances	Ensure same JWT_SECRET in .env on all nodes
Email not sent	SMTP FromEmail ≠ Username	Set FromEmail = Username in config (Office365 enforces this)
docker compose build fails	Node/dotnet version mismatch in Dockerfile	Check Dockerfile FROM versions match project requirements
Bookings disappear on restart	DataSeeder not idempotent	Already fixed — seeder skips if data exists
Docker Hub push 401/403	Invalid DOCKERHUB credentials or token scope	Regenerate the Docker Hub access token and update the GitHub Actions secrets